Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 270 (2025) 692–702

Procedia Computer Science

www.elsevier.com/locate/procedia

29th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2025)

Dynamic Reward-Based Deep Reinforcement Learning Algorithm for UAV Path Planning in Large-Scale Environments

Raja Jarray[a,b,*], Imen Zaghbani[a,c], Soufiene Bouallègue[a,b]

[a]*Research Laboratory in Automatic Control (LARA), National Engineering School of Tunis (ENIT), University of Tunis El Manar, 1002 Tunis, Tunisia*

[b]*High Institute of Industrial Systems of Gabès, University of Gabès, 6072 Gabès, Tunisia*

[c]*National Engineering School of Gabès, University of Gabès, 6029 Gabès, Tunisia*

Abstract

Path planning for Unmanned Aerial Vehicles (UAV) is a vital component of navigation in robotics. The reinforcement Q-learning algorithm enhances path planning for drones but suffers from the need for a large Q-value table and challenges in complex navigation situations. By integrating deep learning with reinforcement one, these shortcomings can be addressed. In this paper, a Deep Q-Network (DQN) model is developed and trained to estimate the drone's state-action value function. In this work, the flight space is represented by a grid of cells, which are then encoded to convert environmental information into a new input format suitable for the DQN model. Normalizing state inputs enhances the stability and convergence of the proposed DQN algorithm by ensuring comparability of features across different scales. Besides, a new dynamic reward function is established based on the distance between the drone's current position and its destination. Simulation results and discussion illustrate the effectiveness of the proposed DQN-based approach for collision-free path planning of UAV in complex environments.

Keywords: Unmanned Aerial Vehicles, path planning, deep Q-learning (DQN), dynamic reward, evaluation and target Q-network.

1. Introduction

As science and technology progress, UAV have emerged as a new trend in development. Path planning, serving as a fundamental aspect of unmanned systems, stands out as a key technology for enabling UAV to achieve intelligence [1]. Path planning aims to compute an optimal trajectory from a start to a goal destination while avoiding obstacles, a challenge that has inspired extensive research and diverse optimization strategies. Despite their usefulness, the heuristic and metaheuristic algorithms exhibit notable limitations when applied to large-scale and complex environments. These approaches are prone to getting trapped in local optima, especially in cluttered or high-dimensional

* Corresponding author. Tel.: +216 28753878
E-mail address: raja.jarray@enit.utm.tn

1877-0509
10.1016/j.procs.2025.09.189

spaces. In contrast, learning-based methods, particularly those based on Reinforcement Learning (RL), offer a promising alternative by enabling agents to learn optimal behaviors through direct interaction with their environment [2]. Among them, Deep Reinforcement Learning (DRL) methods, which combine deep neural networks with RL principles, have shown great potential in solving high-dimensional decision-making problems. The DRL stands out for its ability to adapt to complex and large-scale environments without prior knowledge, autonomously updating its policies through interaction with the environment, making it a promising solution for UAV path planning.

The main contribution in this paper is to propose an effective DRL approach based on the DQN algorithm to tackle the challenge of drone path planning in complex and large-scale environments. The proposed method introduces a novel state encoding technique to mitigate the state space explosion issue encountered in RL. Additionally, a dynamic reward function is designed, leveraging real-time environmental information to adaptively guide the UAV towards its target while avoiding obstacles. The input states are normalized to enhance stability and convergence during training. The carried out simulation experiments and performance analyses demonstrate the efficiency and superiority of the proposed DQN algorithm compared to other state-of-the-art algorithms such as SSA, GWO, PSO, and Q-learning.

The structure of the paper is outlined as follows: Section 2 presents the related work, providing an overview of existing approaches to UAV path planning. Section 3 introduces the proposed DQN-based algorithm for UAV path planning, where the related problem is formulated as a Markov Decision Process (MDP). In Section 4, we showcase and analyze results obtained from diverse flight scenarios with escalating complexity, aiming to demonstrate the effectiveness of the proposed DRL method in obstacles avoidance. Finally, Section 5 concludes the paper and outlines future research directions.

2. Related works

Recent research on UAV path planning has explored a wide range of techniques, which can be broadly categorized into conventional algorithms, metaheuristics optimization, and learning-based approaches. Table 1 summarizes representative studies from each category.

Table 1. Summary of the main UAV path planning approaches.

Categories	Year	References	Description of the main topic and used technique
Conventional methods	2015	[3]	Artificial Potential Field (APF) for attractive/repulsive navigation.
	2016	[4]	RRT applied to generate paths in cluttered 3D environments.
	2021	[5]	A* algorithm for finding shortest paths in static UAV environments.
	2021	[6]	Voronoï diagrams used for obstacle-free safe paths.
	2023	[7]	D* algorithm for solving UAV path planning.
Metaheuristics	2020	[8]	Multi-Verse Optimizer (MVO) for global UAV path optimization.
	2020	[9]	Particle Swarm Optimization (PSO) used for obstacle avoidance.
	2021	[10]	MVO for multi-objective UAV path planning problem.
	2022	[11]	Grey Wolf Optimizer (GWO) applied to solve the UAV path planning.
	2022	[12]	Parallel cooperative co-evolutionary GWO for large-scale UAV path planning.
Reinforcement learning	2018	[13]	Q-learning for dynamic path planning under uncertainty.
	2020	[14]	3D path planning using Q-learning with heuristic reward and replay.
	2023	[15]	Q-learning minimizing energy in post-disaster UAV deployment.
Deep reinforcement learning	2022	[16]	Heuristic DQN using weighted reward function for better convergence.
	2023	[17]	DQN with collaborative energy-aware navigation in dynamic settings.
	2025	[18]	DRL with 3D spatial encoding for compact environment representation.
	2025	[19]	Dueling DQN integrated with APF for guided obstacle-aware learning.

3. Path planning problem formulation

A method for modeling the UAV flight environment involves representing the airspace as a three-dimensional grid, where each cell corresponds to a specific volume of space. As shown in Fig.1, the 3D grid is composed of fixed-size cubic cells, set according to UAV dimensions and safety margins. Specifically, to avoid collisions, the UAV is modeled

as a sphere circumscribed body with radius R_{uav}. Each cell has a side length equal to R_{uav}. These dimensions are based on the safety margin required for the drone, carefully selected to balance computational efficiency and accuracy. Obstacles within the space are represented by aggregating multiple grid cells, forming a discrete representation of their shape and size. To enhance safety, a buffer zone is added around each obstacle by extending the occupied cells. In Fig. 1, the starting and target points, $S\ (x_S, y_S, z_S)$ and $T\ (x_T, y_T, z_T)$ respectively, are defined as specific grid cells, and the path is represented as a sequence of transitions between adjacent cells. This sampling representation simplifies the identification of feasible paths, particularly in environments with irregularly shaped or clustered obstacles.

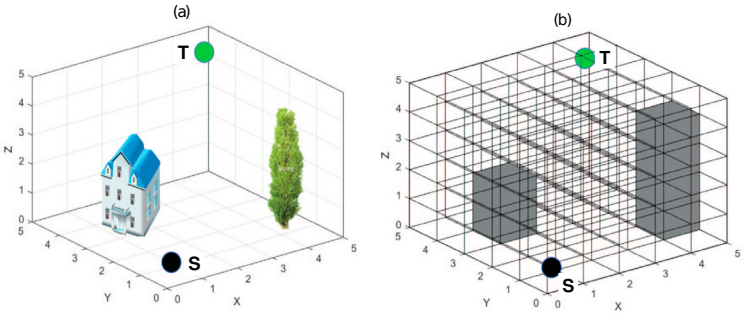

Fig. 1. Flight environment modeling: (a) real environment, (b) environment modeling.

In this study, the UAV is allowed to navigate the environment with all neighboring directions. These directions are strategically defined to ensure both flexibility and safety in the drone's navigation. At the boundaries of the environment, movements are restricted to prevent the UAV from exceeding the predefined operational limits, with all actions directed inward. The diagonal movements are generally prioritized, as they offer a more direct path to the target while minimizing the travel distance. Additionally, when obstacles are detected within the UAV's vicinity, any movement leading to a potential collision is automatically excluded from the set of feasible actions.

4. DQN-based UAV path planning

4.1. DQN design

In DQN framework, the agent explores different actions, receives feedback in the form of rewards, and gradually adjusts its decision-making process to maximize cumulative rewards [20, 21]. This learning process is structured as a MDP. The state space $\mathcal{S}$ represents all possible states $s_t \in \mathcal{S}$. The action space $\mathcal{A}$ defines the set of actions $a_t \in \mathcal{A}$. The reward function $\mathcal{R}$ assigns a value to each action a_t taken in a given state s_t, i.e. $\mathcal{R}(s_t, a_t) = r_t$. The transition probability T dictates the likelihood of moving from one state to another based on a chosen action. Additionally, a discount factor γ is introduced to balance immediate and future rewards.

The proposed DQN learning model utilizes two separate Q-networks: evaluation $Q_E\ (s_t, a_t|\theta_E)$ for action selection and target $Q_T\ (s_{t+1}, a_{t+1}|\theta_T)$ for stabilizing updates.
At each iteration, Q_E first perceives the current states representation, estimates the Q-values, and enables the research agent to select an action by following the ε-greedy policy [17, 22]:

$$a_t = \begin{cases} \arg\max\limits_{a} Q_E(s_t, a_t|\theta_E), & \text{with probability } 1-\varepsilon \\ \text{random action}, & \text{with probability } \varepsilon \end{cases} \tag{1}$$

Once an action is executed, the agent receives a reward forming an experience tuple denoted as $e_t = (s_t, a_t, r_t, s_{t+1})$. The recorded experience is stored in the replay buffer. When such a buffer is full, the newest records will replace the oldest ones to be stored, as shown in Fig.2. Q_E is updated by randomly sampling κ data from the replay buffer. The relative weighting coefficients θ_E are update to minimize the loss function, which is represented by the MSE criterion between target and estimated Q-values, as follows:

$$\mathcal{L}(\theta_E) = E\left[(r_t + \gamma \max Q_T\ (s_{t+1}, a_{t+1}|\theta_T) - Q_E\ (s_t, a_t|\theta_E))^2\right] \tag{2}$$

where $E[.]$ is the expectation operator, r_t the reward value,$\gamma \in [0, 1]$ the discount factor,$Q_T(s_{t+1}, a_{t+1}|\theta_T)$ denotes the predicted value, and $Q_E(s_t, a_t|\theta_E)$ is the target one.

After a predefined number of episodes, the Q_E parameters are periodically synchronized with those of the Q_T one to maintain learning stability. This approach mitigates the issue of excessive data correlation during updates in a single network, thereby enhancing the algorithm convergence. The proposed training process employs the stochastic gradient descent algorithm to update the network parameters as follows:

$$\Delta\theta_E = [r_t + \gamma \max Q_T(s_{t+1}, a_{t+1}|\theta_T) - Q_E(s_t, a_t|\theta_E)] \frac{\delta Q_E(s_t, a_t|\theta_E)}{\delta\theta_E} \tag{3}$$

4.2. Proposed adaptive reward function

The design of a reward function plays a crucial role in the effectiveness of RL algorithms. A well-structured function not only guides the agent toward rapid convergence but also enhances its ability to navigate complex environments [23]. In this study, a adaptive reward function has been carefully designed to incorporate information about the drone's current position and its destination as follows:

$$\mathcal{R} = \begin{cases} 1 & \text{if } s_{t+1} \text{ is the target node,} \\ -1 & \text{if } s_{t+1} \text{ is the forbidden node,} \\ \frac{d_3}{d_1+d_2} & \text{otherwise.} \end{cases} \tag{4}$$

where the $d_1 = \sqrt{(x_{t+1} - x_t)^2 + (y_{t+1} - y_t)^2 + (z_{t+1} - z_t)^2}$, $d_2 = \sqrt{(x_{t+1} - x_T)^2 + (y_{t+1} - y_T)^2 + (z_{t+1} - z_T)^2}$ and $d_3 = \sqrt{(x_t - x_T)^2 + (y_t - y_T)^2 + (z_t - z_T)^2}$, are the Euclidian distances, (x_t, y_t, z_t) present the UAV coordinates in the current state, and $(x_{t+1}, y_{t+1}, z_{t+1})$ are those in the next state .

4.3. Network structure

As shown in Fig.3, the neural network input layer processes the current states representation, encapsulating the available environmental information and the UAV's status. The hidden layers consist of two 3D convolutional ones, each followed by a Rectified Linear Unit (ReLU) type of activation function [28]. The first 3D convolutional layer uses 64 filters with a Kernel size of $(5 \times 5 \times 5)$, a stride of 1 in each dimension, and zero-padding to preserve spatial dimensions. The second one applies 32 filters with the same Kernel size and stride, allowing deeper spatial feature extraction while maintaining spatial consistency.

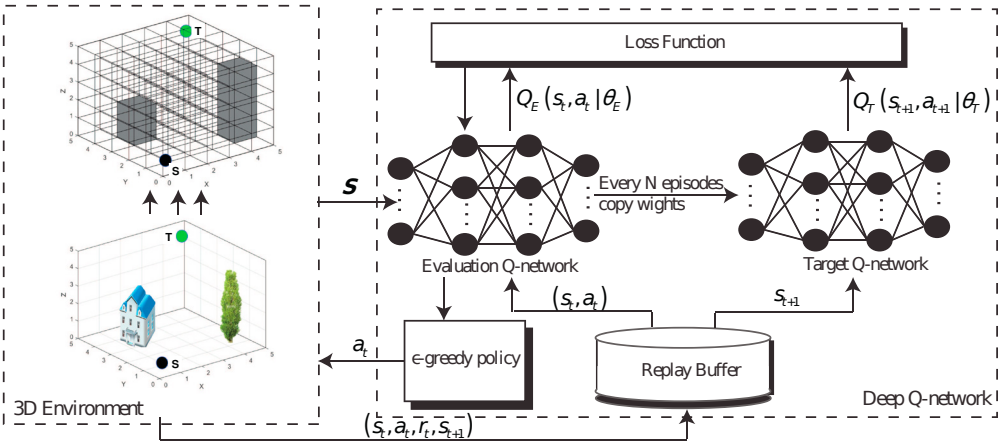

Fig. 2. Proposed DQN-based path planning methodology.

After feature extraction, two fully connected layers each further process the learned representations to enhance decision-making. The first fully connected layer has 256 neurons and the second one has 256 neurons. The output layer consists of a number of neurons, each corresponding to a possible UAV's action, i.e. 26 neurons. Meanwhile, a regression layer is used to optimize Q-values estimation in the RL process. Finally, a pseudo-code for the proposed DQN-based path planning method is presented in Algorithm 1.

Algorithm 1. Proposed DQN algorithm

Input: observation of actual states $\mathcal{S}$.

Output: action a_t

Step 1: Initialize all algorithm parameters γ, ε, κ, max $\_episode$, max $\_step$, θ_E and θ_T, $\theta_T = \theta_E$.

Step 2: **For** episodes=1, max $\_episode$ **do**

While t is less than max $\_step$ **do**

Select $a_t \in \mathcal{A}$ using the ε-greedy policy and execute such an action.

Receive an immediate reward r_t according to Eq. (4), and next state s_{t+1}.

Store the experience $e_t = (s_t, a_t, r_t, s_{t+1})$ in the replay buffer.

Sample a random mini-batch of κ experiences $\{e_1, e_2, \ldots, e_\kappa\}$ from the replay buffer.

Construct target values y_t as:

$$y_t = \begin{cases} r_t & \text{if target node} \\ r_t + \gamma \max Q_T(s_{t+1}, a_{t+1}|\theta_T) & \text{otherwise} \end{cases}$$

Compute the loss function $\mathcal{L}(\theta_E)$ according to Eq. (2).

Update the weighting coefficients θ_E according to Eq. (3).

Update the target network parameters θ_T per certain steps $\theta_T \leftarrow \theta_E$.

Increment the processing time $t = t + 1$.

End While.

End For.

5. Simulation results and discussion

Three different scenarios with increasing complexity for the proposed DQN-based path planning are considered as shown in Table 2. The key parameters used during the model's training phase are summarized in Table 3.

Table 2. Flight environment parameters.

Scenarios	Starting point [km]	Target point [km]	Static obstacles' number
1	(0.5, 0.5, 0.5)	(6.5, 6.5, 3.5)	6
2	(0.5, 0.5, 0.5)	(13.5, 11.5, 3.5)	10
3	(0.5, 0.5, 0.5)	(19.5, 19.5, 3.5)	24
4	(0.5, 0.5, 0.5)	(24.5, 24.5, 24.5)	40

Table 3. Parameters setting of the proposed DQN algorithm.

Parameters	Description	Values
max $\_episode$	Maximum of algorithm episodes	2000
max $\_step$	Maximum of algorithm steps	10000
γ	Discount factor	0.8
ε	Exploratory factor	0.9
κ	Batch size	32
S_M	Replay buffer size	2000

To evaluate the effectiveness of the proposed DQN-based path planner, a comparative study is conducted with various state-of-the-art algorithms, namely Q-learning [20, 24], Grey Wolf Optimizer (GWO) [25], Salp Swarm Algorithm (SSA) [26], and Particle Swarm Optimizer (PSO) [27], across the four scenarios considered. Table 4 provides

a summary of the results obtained from 20 independent executions for all competing algorithms. These findings highlight the most commonly used performance metrics namely Straightness Line Rate (SLR), Computational Time (CT), Path Length (PL) and Success Rate (SR). To assess the consistency of these results, standard deviation (STD) values are computed for each reported solver. Figures 3 and 4 present the SLR results, showcasing the variations in performance across different iterations. Figures 5 and 6 illustrate the box-and-whisker distributions of SLR and CT metrics, allowing for the evaluation of performance dispersion and reproducibility.

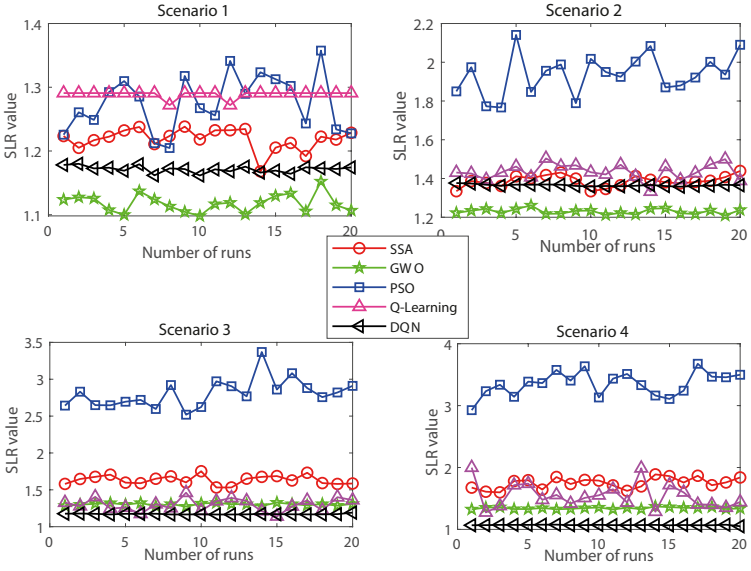

Fig. 3. Comparison of path straightness of reported algorithms in navigation scenarios.

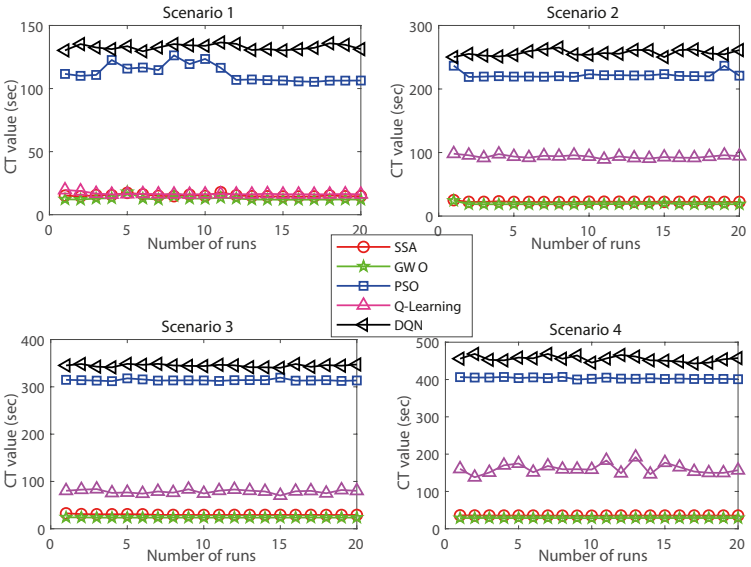

Fig. 4. Comparison of computational time of reported algorithms in navigation scenarios.

In terms of SLR performance, the results indicate that the DQN algorithm demonstrates high reliability, with a low STD. Findings indicate minimal dispersion, meaning high straightness performance across different executions and robustness under variations in initial conditions and stochastic features in the learning process. In contrast, the other algorithms exhibit poor reproducibility, as evidenced by their higher STD and wider interquartile range in the

box-and-whisker plots, reflecting a less predictable performance. This fluctuation could be due to their sensitivity to the trade-offs between exploration and exploitation capabilities. This plots also show that the narrower interquartile range for DQN confirms the regularity of its straightness performance, while the presence of outliers in other algorithms highlights occasional larger deviations. However, GWO outperforms its counterparts in terms of CT performance in most test scenarios. On the other hand, the proposed DQN algorithm requires a long execution time, primarily due to the high computational cost of training the deep neural network. Although, DQN requires a longer CT measures, it offers significant advantages in terms of path quality, success rate, and adaptability. Unlike GWO which is faster but limited to static optimization, DQN learns from interactions with the environment, allowing it to generalize better and handle complex scenarios more effectively. Moreover, in practical applications, the computation time is not always a critical factor, especially when path planning is performed offline prior to the UAV's mission. In such cases, the quality and robustness of the generated path take precedence over computation time.

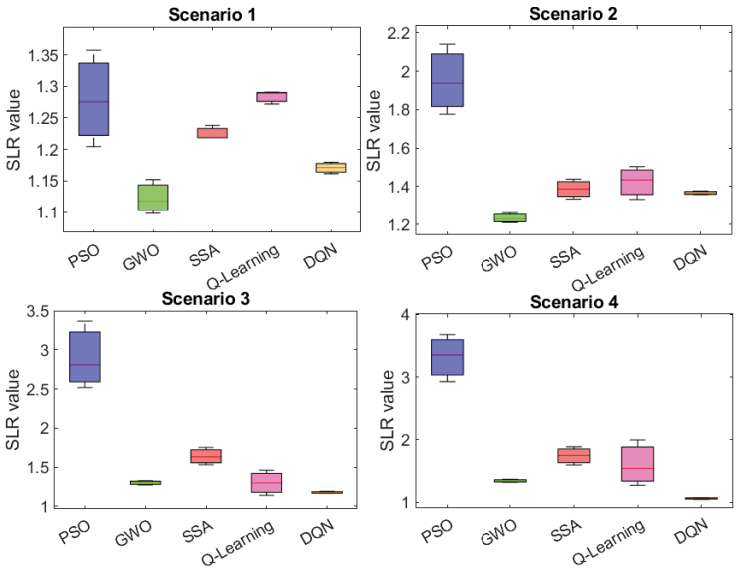

Fig. 5. Box-and-whisker plots of data distribution for SLR metrics.

Figures 7 to 10 illustrate the ability of the competing algorithms to generate collision-free paths while ensuring itineraries shortness, straightness and smoothness. In the first scenario, all algorithms produce feasible and collision-free paths. However, in scenarios 2, 3, and 4, the metaheuristics GWO, SSA, and PSO generate paths that cross threat zones due to dimensionality limitations, making them unsuitable for large-scale problems. In contrast, learning algorithms, namely Q-learning and DQN, successfully avoid these critical areas. Nevertheless, the results reveal a notable difference between the paths generated by the proposed DQN and Q-learning. While both learning approaches effectively avoid threat zones, the path obtained by Q-learning exhibits significant fluctuations, making the path less smooth and potentially less efficient in terms of tracking accuracy and stability. In contrast, the DQN algorithm produces a smoother and straighter path, with a more gradual transition between waypoints. This difference can be attributed to enhanced DQN ability to generalize decisions through its deep neural network, enabling it to learn more robust and coherent strategies in large-scale environments. On the other hand, the proposed DQN algorithm remains attractive due to its superior performance in terms of SR, PL and SLR, as well as the adaptability to complex environments. In contrast, DQN continuously improves its decision-making policy through its interactions with the environment, enabling it to find safer and more efficient paths, especially in high-dimensional and obstacle-dense spaces. These advantages are particularly evident in complex environments where heuristic methods tend to fall into local optima.

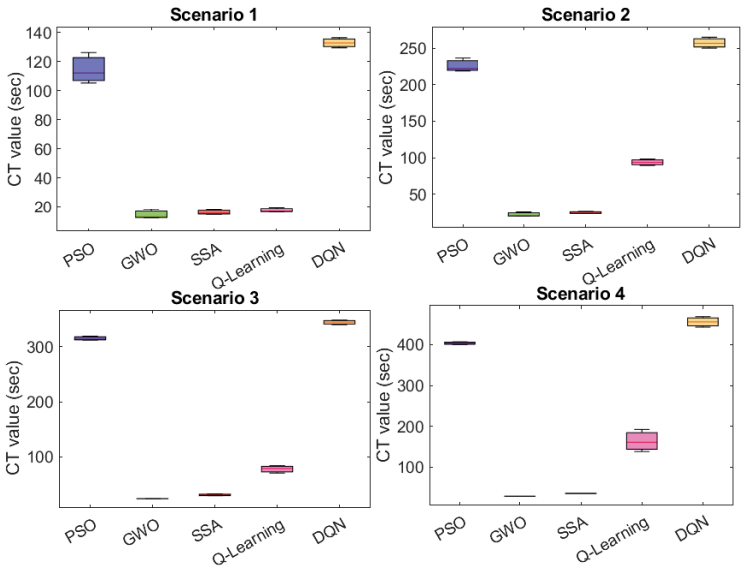

Fig. 6. Box-and-whisker plots of data distribution for CT metrics.

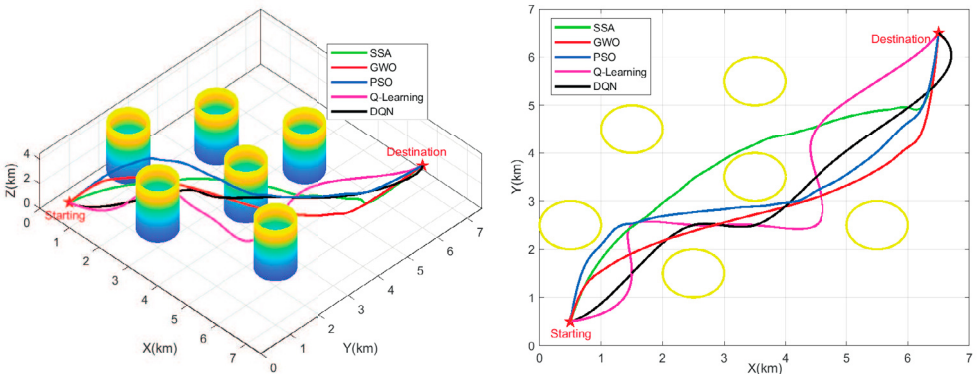

Fig. 7. Results of collision-free path planning within Scenario 1.

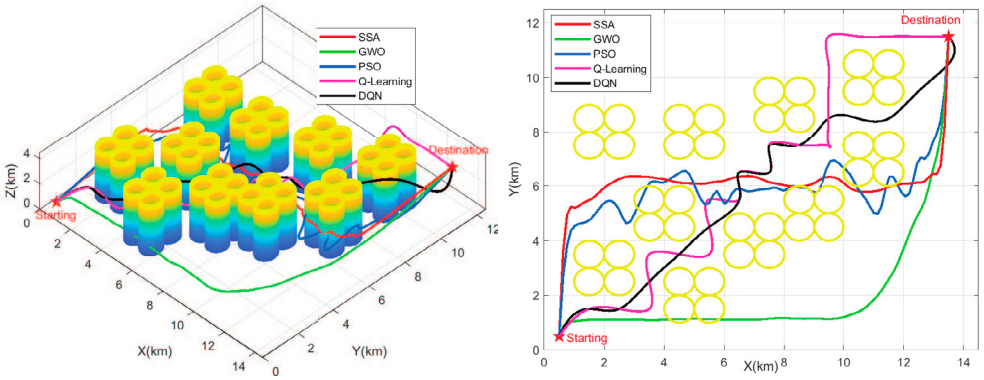

Fig. 8. Results of collision-free path planning within Scenario 2.

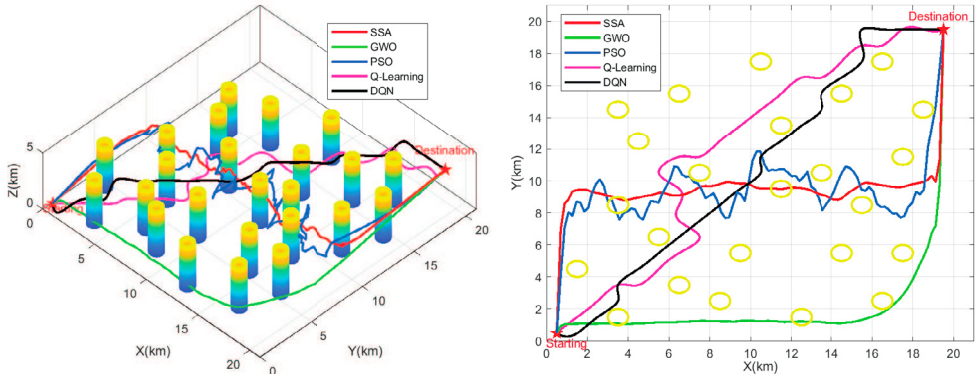

Fig. 9. Results of collision-free path planning within Scenario 3.

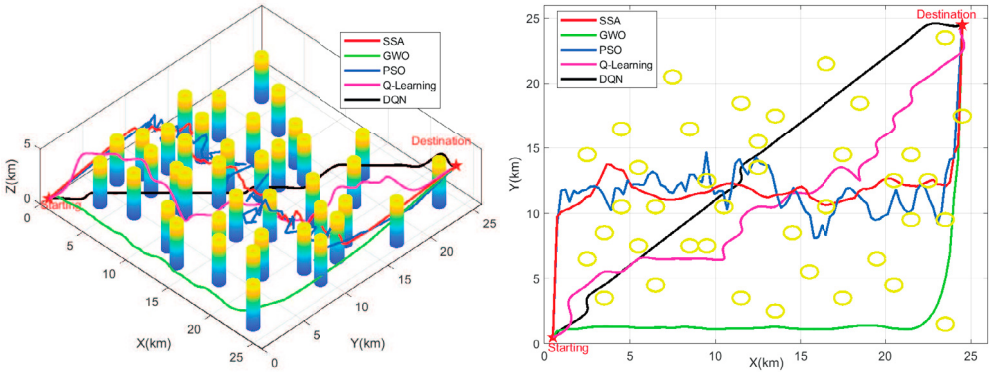

Fig. 10. Results of collision-free path planning within Scenario 4.

6. Conclusion

This study presented a DQN-based approach for UAV path planning, addressing the limitations of traditional Q-learning in large-scale environments. The proposed method introduces a structured encoding of the environment into a grid format, reducing the state space explosion issue and improving learning efficiency. The normalization of state inputs further contributed to the robustness of the learning process. Additionally, a dynamic reward function was designed to incorporate real-time environmental variations, enhancing decision-making adaptability and convergence stability. Simulation results demonstrate the effectiveness of the suggested DQN-based path planning strategy in generating short, smooth and collision-free paths in terms of SR, SLR and PL performance metrics, confirming its potential for real-world UAV navigation applications. While the DQN algorithm incurs a higher computational cost, this aspect is often secondary in practical UAV operations where path safety, success rate, and adaptability are of greater importance.

Future work will focus on parallelizing DQN training to improve efficiency and reduce execution time, while also exploring multi-agent systems to enhance the adaptability and robustness of the UAV path planning strategy.

References

[1] Zhou, Yuquan, Li Yan, Yaxi Han, Hong Xie, and Yinghao Zhao. (2025) "A Survey on the Key Technologies of UAV Motion Planning." *Drones* **9** (3): 194.

[2] Kober, Jens, J. Andrew Bagnell, and Jan Peters. (2013) "Reinforcement learning in robotics: A survey." *The International Journal of Robotics Research* **32** (11): 1238–1274.

[3] Budiyanto, Almira, Adha Cahyadi, Teguh Bharata Adji, and Oyas Wahyunggoro. (2015) "UAV obstacle avoidance using potential field under dynamic environment." In *Proceedings of the 2015 Conference on Control, Electronics, Renewable Energy and Communications*, Bandung, Indonesia, pp. 187–192.
[4] Ma, Rong, Wen Ma, Xiaolong Chen, and Jia Li. (2016) "Real-time obstacle avoidance for fixed-wing vehicles in complex environment." In *Proceedings of the IEEE Chinese Guidance, Navigation and Control Conference*, Nanjing, China, pp. 498–502.
[5] Mandloi, Dilip, Rajeev Arya, and Ajit K. Verma. (2021) "Unmanned aerial vehicle path planning based on A* algorithm and its variants in 3D environment." *International Journal of System Assurance Engineering and Management* **12** (5): 990–1000.
[6] Huang, Sheng-Kai, Wen-June Wang, and Chung-Hsun Sun. (2021) "A path planning strategy for multi-robot moving with path-priority order based on a generalized Voronoi diagram." *Applied Sciences* **11** (20), https://doi.org/10.3390/app11209650.
[7] Chattaoui, Samah, Raja Jarray, and Soufiene Bouallègue. (2023) "Comparison of A* and D* algorithms for 3D path planning of unmanned aerial vehicles." In *Proceedings of the 2023 IEEE International Conference on Artificial Intelligence and Green Energy*, Sousse, Tunisia, doi: 10.1109/ICAIGE58321.2023.10346511.
[8] Jarray, Raja, and Soufiene Bouallègue. (2020) "Multi-verse algorithm-based approach for multi-criteria path planning of unmanned aerial vehicles." *International Journal of Advanced Computer Science and Applications* **11** (11): 324–334.
[9] Shao, Shikai, Yu Peng, Chenglong He, and Yun Du. (2020) "Efficient path planning for UAV formation via comprehensively improved particle swarm optimization." *ISA Transactions* **97** (1): 415–430.
[10] Jarray, Raja, Mujahed Al-Dhaifallah, Hegazy Rezk, and Soufiene Bouallègue. (2021) "Path planning of quadrotors in a dynamic environment using a multi-criteria multi-verse optimizer." *Computers, Materials and Continua* **69** (1): 2159–2180.
[11] Jarray, Raja, and Soufiene Bouallègue. (2022) "Path planning strategy for unmanned aerial vehicles based on a grey wolf optimizer." *International Journal of Intelligent Engineering Informatics* **9** (6): 551–577.
[12] Jarray, Raja, Mujahed Al-Dhaifallah, Hegazy Rezk, and Soufiene Bouallègue. (2022) "Parallel cooperative co-evolutionary grey wolf optimizer for path planning problem of unmanned aerial vehicles." *Sensors* **22** (5): 1–29.
[13] Cui, Jun-hui, Rui-xuan Wei, Zong-cheng Liu, and Kai Zhou. (2018) "UAV motion strategies in uncertain dynamic environments: A path planning method based on Q-learning strategy." *Applied Sciences* **8** (11), https://doi.org/10.3390/app8112169.
[14] Xie, Ronglei, Zhijun Meng, Yaoming Zhou, Yunpeng Ma, and Zhe Wu. (2020) "Heuristic Q-learning based on experience replay for three-dimensional path planning of the unmanned aerial vehicle." *Science Progress* **103** (1), https://doi.org/10.1177/0036850419879024.
[15] Souto, Anderson, Rodrigo Alfaia, Evelin Cardoso, Jasmine Araújo, and Carlos Francês. (2023) "UAV path planning optimization strategy: Considerations of urban morphology, microclimate, and energy efficiency using Q-Learning algorithm." *Drones* **7** (2), https://doi.org/10.3390/drones7020123.
[16] Yao, Jiangyi, Xiongwei Li, Yang Zhang, Jingyu Ji, Yanchao Wang, and Yicen Liu. (2022) "Path planning of unmanned helicopter in complex environment based on heuristic deep Q-network." *International Journal of Aerospace Engineering* **2022**, https://doi.org/10.1155/2022/1360956.
[17] Guo, Zhihui, Hongbin Chen, and Shichao Li. (2023) "Deep reinforcement learning-based UAV path planning for energy-efficient multitier cooperative computing in wireless sensor networks." *Journal of Sensors* **2023**, https://doi.org/10.1155/2023/2804943.
[18] Wang, Zhipeng, Soon Xin Ng, and Mohammed El-Hajjar. (2025) "A 3D Spatial Information Compression Based Deep Reinforcement Learning Technique for UAV Path Planning in Cluttered Environments." *IEEE Open Journal of Vehicular Technology* **6**: 647–661, https://doi.org/10.1109/OJVT.2025.3540174.
[19] Li, Chang, Xiaofeng Yue, Zeyuan Liu, Guoyuan Ma, Hongbo Zhang, Yuan Zhou, and Juan Zhu. (2025) "A modified dueling DQN algorithm for robot path planning incorporating priority experience replay and artificial potential fields." *Applied Intelligence* **55** (6): 1–27.
[20] Powell, Warren B. (2011) "Approximate Dynamic Programming: Solving the Curses of Dimensionality, 2nd Edition." *John Wiley and Sons Inc. Publication*, New Jersey, USA.
[21] Dayan, Peter, and C. J. C. H. Watkins. (1992) "Q-learning," *Machine Learning* **8** (3-4): 279–292.
[22] Chung, Jonatha. (2013) "Playing Atari with deep reinforcement Learning." *Comput. Ence.* **21**: 351–362.
[23] Wang, Baolai, Shengang Li, Xianzhong Gao, and Tao Xie. (2021) "UAV swarm confrontation using hierarchical multiagent reinforcement learning." *International Journal of Aerospace Engineering* **2021** (1), https://doi.org/10.1155/2021/3360116.
[24] Zaghbani, Imen, Raja Jarray, and Soufiene Bouallègue. (2024) "Comparative Study of Q-Learning and SARSA Algorithms for UAV Path Planning in 3D Environments." In *Proceedings of the 2024 IEEE 28th International Conference on Intelligent Engineering Systems*, Gammarth, Tunisia, pp. 245–250.
[25] Mirjalili, Seyedali, Seyed Mohammad Mirjalili, and Andrew Lewis. (2014) "Grey wolf optimizer." *Advances in Engineering Software* **69** (1): 46–61.
[26] Mirjalili, Seyedali, Amir H. Gandomi, Seyedeh Zahra Mirjalili, Shahrzad Saremi, Hossam Faris, and Seyed Mohammad Mirjalili. (2017) "Salp swarm algorithm: A bio-inspired optimizer for engineering design problems." *Advances in Engineering Software* **114** (1): 163–191.
[27] Clerc, Maurice, and James Kennedy. (2002) "The particle swarm: explosion, stability, and convergence in a multidimensional complex space." *IEEE Transactions on Evolutionary Computation* **6** (1): 58–73.
[28] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016) "Deep Learning", Cambridge, MA, MIT Press.

Table 4. Path planning results in each scenario.

Algorithms	Metrics	Scenario 1				Scenario 2				Scenario 3				Scenario 4			
		Best	Mean	Worst	STD	Best	Mean	Worst	STD	Best	Mean	Worst	STD	Best	Mean	Worst	STD
PSO	SLR	1.2046	1.2755	1.3574	0.0440	1.7766	1.9381	2.1410	0.1046	2.5191	2.8084	3.3693	0.1951	2.9265	3.3512	3.6758	0.1938
	CT(sec)	105.33	112.23	126.30	6.733	218.96	222.22	236.681	5.084	311.99	314.11	319.41	1.780	400.07	403.42	406.91	2.138
	PL(km)	10.204	11.479	12.216	0.0440	30.720	33.512	37.021	0.1046	68.108	75.930	91.095	0.1951	121.65	139.30	152.80	0.1938
	SR (%)		60				40				35				20		
GWO	SLR	1.0981	1.1179	1.1522	0.0140	1.2095	1.2294	1.2609	0.0137	1.2721	1.3025	1.3268	0.0161	1.3226	1.3426	1.3698	0.0145
	CT(sec)	11.975	12.771	17.973	1.425	18.497	19.063	24.230	1.273	23.465	23.603	24.095	0.173	28.299	28.652	28.901	0.159
	PL(km)	9.8829	10.061	10.3698	0.0140	20.914	21.258	21.802	0.0137	34.393	35.215	35.872	0.0161	54.979	55.810	56.941	0.0145
	SR (%)		75				45				40				25		
SSA	SLR	1.2187	1.2241	1.2382	0.0169	1.3343	1.3871	1.4388	0.0292	1.5320	1.6337	1.7520	0.0620	1.5983	1.7489	1.8883	0.0893
	CT(sec)	14.423	15.231	17.855	0.9470	22.041	22.476	25.046	0.647	29.058	29.690	32.532	0.946	34.873	35.302	35.904	0.214
	PL(km)	10.968	11.016	11.143	0.0169	23.072	23.985	24.879	0.0292	41.420	44.170	47.368	0.0620	66.440	72.700	78.495	0.0893
	SR (%)		70				40				35				25		
Q-Learning	SLR	1.2720	1.2890	1.2908	0.0058	1.3326	1.4346	1.5036	0.0416	1.1395	1.2985	1.4606	0.0852	1.2743	1.5443	1.9979	0.2054
	CT(sec)	16.091	16.583	19.604	0.910	89.294	93.596	98.085	2.266	70.276	78.816	83.805	3.582	138.05	160.71	192.04	13.517
	PL(km)	11.448	11.601	11.617	0.0058	23.042	24.806	25.999	0.0416	30.808	35.107	39.490	0.0852	52.971	64.195	83.051	0.2054
	SR (%)		95				85				80				70		
Proposed DQN	SLR	1.1616	1.1714	1.1799	0.0052	1.3578	1.3651	1.3766	0.0049	1.1655	1.1713	1.1913	0.0063	1.0509	1.0697	1.0766	0.0054
	CT(sec)	129.54	132.88	136.54	2.154	250.25	256.93	265.24	4.546	340.25	344.88	348.56	2.665	442.86	455.59	468.58	7.509
	PL(km)	10.454	10.542	10.619	0.0052	23.478	23.604	23.803	0.0049	31.511	31.668	32.209	0.0063	43.685	44.466	44.753	0.0054
	SR (%)		98				93				88				85		